Practical-11

Aim: To implement the Value Iteration or Policy-Based Decision-Making algorithm in a Grid-World environment to find the optimal path and make decisions based on rewards and penalties.

CODE:

```
transitions ={
    'A' : {'Right':'B','Down':'C'},
    'B' : {'Left':'A','Down':'G'},
    'C' : {'Right':'G','Right':'A'},
}
rewards ={
    'A': -1,
    'B': -1,
    'C': -1,
    'G': 10
}
gamma =0.9

values ={
    'A':0,
    "B":0,
    'C':0,
    'G':0
}
print(values)

for i in range(5):
    new_values = values.copy()

    for state in ['A', 'B', 'C']:
        possible_values = []

        for action, next_state in transitions[state].items():
            value = rewards[next_state] + gamma * values[next_state]
            possible_values.append(value)
```

```
    new_values[state] = max(possible_values)
    values = new_values
  print("Final values:")
  print(values)

  policy = {}
  for state in ["A",'B','C']:
    best_action =None
    best_value = -100
    for action, next_state in transitions[state].items():
      value =rewards[next_state] + gamma *values[next_state]

      if value > best_value:
        best_value = value
        best_action =action
    policy[state] = best_action
  print('Best policy:')
  print(policy)

    for state, action in policy.items():
      print("Fron",state,"Choose",action)
```

OUTPUT:

```
{'A': 0, 'B': 0, 'C': 0, 'G': 0}

Final values:
{'A': 8.0, 'B': 10.0, 'C': 6.2, 'G': 0}

Best policy:
{'A': 'Right', 'B': 'Down', 'C': 'Right'}

Fron A Choose Right
Fron B Choose Down
Fron C Choose Right
```

Conclusion:

Thus, the Value Iteration or Policy-Based Decision-Making algorithm was successfully implemented in a Grid-World environment. The agent learned to choose the best actions to reach the goal while maximizing rewards. This practical helped us understand reinforcement learning and sequential decision-making.